EEN1095 Git Repository and Development Record

Student Name	Darshan Mahadeva Naika	**Student ID**	A00090581
Project Title	Design and Evaluation of Low-Sensitivity Digital Filters Implemented in Python Under Varying Numerical Precision Levels	**Supervisor**	Martin Collier
Implementation Period	Week 3 - Week 4	**Version Date**	22/05/2026
Link to the Repository	https://github.com/darshanmahadevanaika2-sudo/EEN1095-Low-Sensitivity-Digital-Filters	**Shared to Supervisor**	Not shared

Guidance: Maintain this as one cumulative document throughout EEN1095. Update it every two weeks and do not remove earlier records. Use it to summarise sustained implementation and development work evidenced through your Git repository or equivalent version-controlled development record. Each entry should summarise the development work completed, the principal files or components affected, and the key evidence of progress. Where applicable, relevant screenshots of the Git commit history should also be provided.

Development Record:

Weeks 1-2	
Main implementation / development work completed	Python multi-precision evaluation framework development. Set up Python 3.11 environment with NumPy, SciPy, and mpmath. Implemented filter_design.py for IIR and FIR filter design using SciPy. Implemented metrics.py defining all five performance metrics. Implemented experiment.py as the main experimental runner. Validated precision control for float16, float32, float64 via NumPy dtype and mpmath via mp.dps=50. Ran pilot experiments for Direct-Form IIR filters at orders 10 and 20 across all four precision levels.
Key files, folders, or components added / updated	filter_design.py — Core filter design and precision conversion module: design_iir_filter(), design_fir_filter(), convert_to_precision(), get_poles_from_a(), compute_freq_response(). metrics.py — Five

	performance metrics module: pole_displacement(), stability_margin(), sensitivity_norm(), freq_response_deviation(), roundoff_noise(). experiment.py — Main experimental evaluation runner: run_iir_experiment(), run_fir_experiment(), run_all_experiments(). results.json — Pilot experimental results dataset for Direct-Form IIR configurations across all precision levels.
Evidence of versioning / commits / iterations	Git repository not yet created — code files are locally version tracked. Initial commit planned once repository is set up in Weeks 3-4. Current iteration: v0.1 — framework setup and pilot testing complete. Three main Python modules implemented and validated: filter_design.py, metrics.py, experiment.py. Pilot results for Direct-Form IIR filters at orders 10 and 20 stored in results.json.
Testing, debugging, or validation carried out	Precision control validation: confirmed float16, float32, float64 dtype selection via NumPy astype() correctly reduces coefficient precision. mpmath mp.dps=50 confirmed providing 50 decimal digit precision. Filter design validated against known analytical frequency responses. Metrics validated on order-10 Butterworth filter. Key finding confirmed: Direct-Form order-20 filters become UNSTABLE at float16 — poles breach the unit circle. All FIR filters remain stable at all precision levels.
Problems encountered and how they were addressed	Minor issue: mpmath coefficient conversion required handling via mpmath.mpf(str(c)) to preserve exact decimal precision during conversion from SciPy float64. Resolved successfully. Minor issue: NumPy overflow warnings during float16 sensitivity norm computation. Resolved by wrapping in try-except and filtering expected overflow warnings.
Next development steps	Weeks 3-4: Create Git repository and push all current framework files. Implement Ladder filter Python module based on Bruton 1975 [1] using cascaded lossless two-port sections. Begin Wave Digital Filter implementation based on Fettweis 1986 [2] and Yli-Kaakinen 2006 [9]. Validate Ladder and WDF implementations against Direct-Form baseline at float64. Run full order-30 experiments for all four IIR filter types across all precision levels.

Weeks 3-4	
Main implementation / development work completed	Formalized the mathematical definition of filter sensitivity pole displacement, stability margin, sensitivity norm, frequency response deviation, and roundoff noise. Implemented and ran the complete Direct-Form IIR baseline evaluation across all 4 precision levels (float16, float32, float64, mpmath) for Butterworth, Chebyshev I/II, and Elliptic filters at orders 10, 20, and 30. Confirmed Direct-Form instability at low precision as the baseline for Ladder filter comparison. Studied the Lossless Discrete Integrator (LDI) transformation from the s-plane to the z-plane as the mathematical foundation for the Ladder filter implementation.
Key files, folders, or components added / updated	src/direct_form_evaluation.py — Complete Direct-Form IIR evaluation module covering all filter types, orders, and precision levels with all 5 metrics. src/filter_design.py — Updated comments and documentation. src/metrics.py — Updated comments and documentation. src/experiment.py — Updated comments and documentation. results/results.json — Pilot results dataset from Direct-Form evaluation.
Evidence of versioning / commits / iterations	Git repository created and connected to GitHub via SSH. Two commits pushed to https://github.com/darshanmahadevanaika2-sudo/EEN1095-Low-Sensitivity-Digital-Filters. Commit 1: 0aeee99 — Initial commit — 5 files, 1589 insertions. Commit 2: 5ae6c83 — Renamed step2 file to direct_form_evaluation.py and updated comments in all files — 4 files changed.
Testing, debugging, or validation carried out	Direct-Form evaluation script run locally on Python 3.13 with NumPy, SciPy, and mpmath. Results validated against expected theoretical behaviour. Key findings confirmed: Butterworth order-20 UNSTABLE at float16 (SM = -0.47). Elliptic order-10 UNSTABLE at float32 (SM = -0.0067). Chebyshev, I order-30 UNSTABLE even at float64 (SM = -0.13). Roundoff noise for Butterworth order-30 at float32 = 7.17×10^{69} — catastrophic divergence. Fixed Python version conflict — VS Code was using Python 3.14 which did not have libraries installed.

	Resolved by running directly with Python 3.13 executable.
Problems encountered and how they were addressed	Problem 1: ModuleNotFoundError: No module named numpy — VS Code was using Python 3.14 but libraries were installed on Python 3.13. Resolved by running the script directly using the correct Python 3.13 path in Command Prompt. Problem 2: GitHub authentication via password failed. Resolved by setting up SSH key authentication — generated ed25519 SSH key, added public key to GitHub, and connected repository via SSH remote URL. Problem 3: curl command failed in PowerShell. Resolved by using Invoke-RestMethod PowerShell equivalent.
Next development steps	Weeks 5-6: Implement the Ladder filter Python module based on Bruton 1975 using the Lossless Discrete Integrator transformation. Derive reflection coefficients α from the normalised analog low-pass prototype. Implement single ladder section difference equation y[n] = x[n] + α × (x[n-1] + y[n-1]). Validate Ladder filter impulse response and frequency response against Direct-Form baseline at float64. Begin cross-precision comparison of Ladder vs Direct-Form at float16, float32, float64, and mpmath.

Weeks 5-6	
Main implementation / development work completed	
Key files, folders, or components added / updated	
Evidence of versioning / commits / iterations	Enter a short summary of commit activity here. Add commit-history screenshots if applicable.
Testing, debugging, or validation carried out	

Problems encountered and how they were addressed	
Next development steps	Outline the planned development tasks for the next two-week period.
Weeks 7-8	
Main implementation / development work completed	
Key files, folders, or components added / updated	
Evidence of versioning / commits / iterations	Enter a short summary of commit activity here. Add commit-history screenshots if applicable.
Testing, debugging, or validation carried out	
Problems encountered and how they were addressed	
Next development steps	Outline the planned development tasks for the next two-week period.

Weeks 9-10	
Main implementation / development work completed	
Key files, folders, or components added / updated	
Evidence of versioning / commits / iterations	Enter a short summary of commit activity here. Add commit-history screenshots if applicable.
Testing, debugging, or validation carried out	

Problems encountered and how they were addressed	
Next development steps	Outline the planned development tasks for the next two-week period.
Weeks 11-12	
Main implementation / development work completed	
Key files, folders, or components added / updated	
Evidence of versioning / commits / iterations	Enter a short summary of commit activity here. Add commit-history screenshots if applicable.
Testing, debugging, or validation carried out	
Problems encountered and how they were addressed	
Next development steps	Outline the planned development tasks for the next two-week period.

Weeks 13-14	
Main implementation / development work completed	
Key files, folders, or components added / updated	
Evidence of versioning / commits / iterations	Enter a short summary of commit activity here. Add commit-history screenshots if applicable.
Testing, debugging, or validation carried out	

Problems encountered and how they were addressed	
Next development steps	Outline the planned development tasks for the next two-week period.